Fundamentals of Pre-Silicon Validation
Winter-2026

Verification of Asynchronous FIFO

UVM Based TB Implementation

Team -10

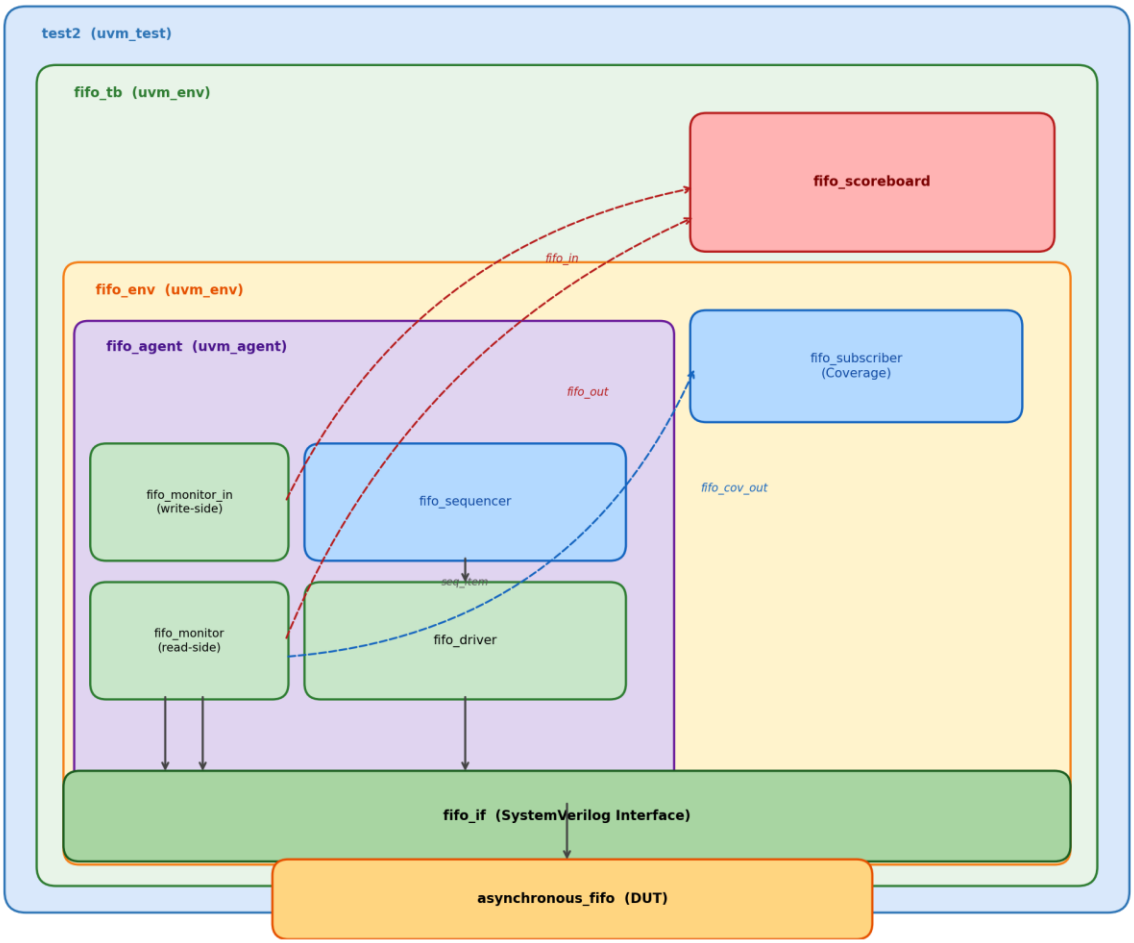
Monesh Kareti – monesh@pdx.edu

Senkathir Mutharasu – senkmuth@pdx.edu

For verifying this Asynchronous FIFO, we employed a UVM-based testbench with the following files:

File	Description
tb_top.sv	Static testbench top module — DUT instantiation, clock/reset generation, run_test()
fifo_pkg.sv	UVM package — imports and compile-order includes for all TB classes
fifo_if.sv	SystemVerilog interface — bundles all DUT ports and auxiliary signals
fifo_packet.sv	UVM sequence item — framed packet transaction with constraints and parity
fifo_sequencer.sv	UVM sequencer — TLM arbitration between sequences and driver
fifo_seqs.sv	UVM sequences — base objection handler and fifo_5_packets burst sequence
fifo_driver.sv	UVM driver — write-side wclk-domain stimulus with frame protocol
fifo_monitor_in.sv	Write-side monitor — reconstructs frames from wclk domain, feeds scoreboard
fifo_monitor.sv	Read-side monitor — drives r_en, reconstructs frames, feeds scoreboard and coverage
fifo_scoreboard.sv	UVM scoreboard — packet-level end-to-end comparison via queue reference model
fifo_subscriber.sv	Coverage subscriber — functional coverage via covergroup sampling
fifo_agent.sv	UVM agent — encapsulates sequencer, driver, and both monitors
fifo_env.sv	Inner UVM environment — agent and subscriber with coverage connection
fifo_tb.sv	Outer UVM environment — wraps fifo_env, holds scoreboard
fifo_test_lib.sv	UVM tests — base_test and test2 (fifo_5_packets sequence)

UVM Testbench Architecture:



1. **tb_top.sv:**

- The testbench top module imports uvm_pkg and fifo_pkg, and includes fifo_tb.sv and fifo_test_lib.sv to complete the UVM component hierarchy.
- A SystemVerilog interface (fifo_if) is instantiated and connected to the DUT (asynchronous_fifo), bundling all input/output signals for clean testbench access.
- Both wclk and rclk are generated at 500 MHz (2 ns period). Reset is staggered: wrst_n is deasserted after 10 write clock cycles, and rrst_n after an additional 20 read clock cycles.
- The virtual interface is registered in uvm_config_db and the UVM test is initiated via run_test("test2").

2. **fifo_pkg.sv:**

- Central UVM package that imports uvm_pkg and includes all testbench class files in the required compile order.
- Compile order: fifo_packet, fifo_sequencer, fifo_seqs, fifo_driver, fifo_monitor, fifo_monitor_in, fifo_subscriber, fifo_scoreboard, fifo_agent, fifo_env.
- Ensures all class definitions are resolved before they are referenced by any other component.

3. **fifo_if.sv:**

- Parameterized SystemVerilog interface with DEPTH=512 and DATA_WIDTH=16.
- Groups all DUT ports — wclk, rclk, wrst_n, rrst_n, w_en, r_en, data_in [15:0], data_out [15:0], full, empty, full_3_4, empty_1_4 — into a single bundle.
- Includes auxiliary signals monstart and drvstart used for transaction recording synchronization between the driver and monitor.
- A virtual interface handle is passed to all testbench components via uvm_config_db.

4. **fifo_packet.sv:**

- UVM sequence item representing one complete FIFO packet using a framed protocol: ID → LENGTH → PAYLOAD[0..N-1] → PARITY.
- id is randc (avoids consecutive repetition); length is constrained > 0; payload[] size is constrained to equal length.
- Parity is automatically computed as the XOR of all payload words in post_randomize().
- write_idle_cycles and read_idle_cycles are constrained to 0 and 4 respectively and registered as UVM_NOCOMPARE to exclude them from scoreboard comparison.
- w_en, r_en, full, and empty fields are non-randomized and populated by the monitor for functional coverage sampling.

5. **fifo_sequencer.sv:**

- Standard UVM sequencer parameterized on fifo_packet.
- Provides the TLM seq_item_export for connection to the driver's seq_item_port.
- No custom arbitration logic; acts as a pass-through between sequences and the driver.

6. **fifo_seqs.sv:**

- fifo_base_seq: base sequence class that manages UVM phase objections — raises in pre_body() via starting_phase and drops in post_body() to control simulation lifetime.
- fifo_5_packets: extends fifo_base_seq and runs 100 iterations, each randomizing a fifo_packet constrained to length == 450, representing the 450-item burst specification.
- Directed sequences for edge case scenarios (test_full, test_empty, test_full_3_4, test_empty_1_4) are planned for future milestones.

7. **fifo_driver.sv:**

- Active UVM driver generating write-side stimulus on the wclk domain. Retrieves the virtual interface from uvm_config_db in connect_phase.
- run_phase forks two threads: get_and_drive() handles packet stimulus, and reset_signals() monitors for asynchronous reset events.
- get_and_drive() waits for posedge wrst_n then loops forever, fetching fifo_packet items from the sequencer and calling send_to_dut().

- send_to_dut() drives the complete frame — id, length, each payload word, then parity — each word gated on negedge wclk iff !full. w_en is asserted for the frame duration and deasserted afterward.
- On negedge wrst_n: drives data_in to high-impedance, clears w_en, and disables send_to_dut. On negedge rrst_n: clears r_en.
- After each packet completes, end_tr() is called for transaction recording and num_sent is incremented. report_phase logs the total packets sent at UVM_LOW verbosity.

8. **fifo_monitor_in.sv:**

- Passive UVM monitor observing the wclk domain. Retrieves the virtual interface from uvm_config_db in connect_phase.
- After detecting posedge wrst_n, continuously reconstructs incoming packet frames by sampling data_in on posedge wclk whenever w_en && !full.
- Captures id, length, payload[] (sized dynamically from length), and parity in frame order.
- Publishes each completed fifo_packet to the scoreboard via the fifo_in TLM analysis port.
- report_phase logs the total number of packets collected.

9. **fifo_monitor.sv:**

- UVM monitor observing the rclk domain that also actively drives r_en. Has two analysis ports: fifo_out (to scoreboard) and fifo_cov_out (to coverage subscriber).
- After posedge rrst_n, asserts r_en on negedge rclk and reads id and length on posedge rclk iff !empty.
- For each payload word: samples data_out, captures w_en/r_en/full/empty into a coverage packet written to fifo_cov_out, deasserts r_en, waits 3 posedge rclk iff !empty cycles, then re-asserts r_en — implementing the 4-cycle read spacing.
- Transaction recording is handled via a fork: collect_packet() runs in parallel with a begin_tr() call triggered on posedge monstart, and end_tr() is called after the frame completes.
- After all payload words, reads parity, deasserts r_en and monstart, then publishes the completed packet to scoreboard via fifo_out. report_phase logs total packets collected.

10. **fifo_scoreboard.sv:**

- Extends uvm_scoreboard with dual analysis imp ports: fifo_in (write-side) and fifo_out (read-side), declared using uvm_analysis_imp_decl macros.
- Tracks three counters: fifo_in_packets_received, fifo_out_packets_received, and mismatch — providing visibility into packet flow and error counts throughout simulation.
- write_fifo_in(): clones each received packet using $cast and pushes it to in_queue, incrementing fifo_in_packets_received.
- write_fifo_out(): pops from in_queue, increments fifo_out_packets_received, and calls packet.compare() — logs UVM_INFO on match, UVM_ERROR on mismatch.
- check_phase(): raises UVM_ERROR if in_queue is not empty at end of simulation, indicating unread packets. report_phase is present as a placeholder for future statistics display.

11. **fifo_subscriber.sv:**

- Extends uvm_subscriber and is connected to the read-side monitor's fifo_cov_out analysis port.
- On each write() call, updates the internal packet handle and calls cg.sample() on the covergroup.
- Implemented coverpoints: w_en, r_en, full, empty (individual); wr_rd_cp cross of {w_en, r_en} with bins idle (2'b00), write_only (2'b10), read_only (2'b01), both (2'b11).
- Coverage bins for full_3_4, empty_1_4, and occupancy ranges are planned for MS5.

12. **fifo_agent.sv:**

- Extends uvm_agent and encapsulates the sequencer, driver, and both monitors (monitor_out for read-side, monitor_in for write-side).
- is_active is registered with uvm_field_enum in the factory macro (uvm_component_utils_begin/end), making it configurable from the test level via uvm_config_db without modifying the agent.
- build_phase always creates both monitors regardless of active/passive mode. Sequencer and driver are only created when is_active == UVM_ACTIVE.
- connect_phase connects driver.seq_item_port to sequencer.seq_item_export when active.

13. **fifo_env.sv:**

- Inner UVM environment parameterized with DATA_WIDTH=16. Instantiates fifo_agent and fifo_subscriber in build_phase.
- connect_phase connects agent.monitor_out.fifo_cov_out to subscriber.analysis_export, establishing the coverage data path.

14. **fifo_tb.sv:**

- Outer UVM environment (uvm_env) that instantiates fifo_env and fifo_scoreboard in build_phase.
- connect_phase establishes scoreboard connections: env.agent.monitor_out.fifo_out to scoreboard.fifo_out, and env.agent.monitor_in.fifo_in to scoreboard.fifo_in.
- Placing the scoreboard at this outer level keeps comparison logic independent of the coverage infrastructure inside fifo_env.

15. **fifo_test_lib.sv:**

- base_test: creates fifo_tb, enables transaction recording globally, prints UVM topology at end_of_elaboration_phase, and sets a 20 µs drain time in run_phase.
- test2: extends base_test and configures fifo_5_packets as the default sequence on the sequencer (tb.env.agent.sequencer.run_phase) via uvm_config_wrapper.
- Additional directed test classes for edge case scenarios will extend base_test in future milestones.